

A Practical Guide to Batch normalization

TOPIC -- BATCH NORMALIZATION

$$\hat{X}_i = \frac{X_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}$$

$$y_i = \gamma \hat{X}_i + \beta$$

-- 01 --

$\|\nabla_{\tilde{w}} f(\tilde{w}; T_d)\|_2 \leq (1 - \mu_L)^2 T_d \Phi^2(\rho(w_0) - \rho^*) + 2 - T_s \zeta |b_t(0) - a_t(0)| \mu^2$ $\{\displaystyle \|\nabla_{\tilde{w}} f(\tilde{w}; T_d)\|_2 \leq (1 - \frac{\mu}{L})^2 T_d \Phi^2(\rho(w_0) - \rho^*) + 2 - T_s \zeta |b_t(0) - a_t(0)| \mu^2\}$, such that the algorithm is guaranteed to converge linearly. Although the proof stands on the assumption of Gaussian input, it is also shown in experiments that GDNP could accelerate optimization without this constraint.

-- 02 --

$\min_{\tilde{W}, \Theta} (f_{NN}(\tilde{W}, \Theta) = E_{y, x} [l(-y F_x(\tilde{W}, \Theta))])$ $\{\displaystyle \min_{\{\tilde{W}, \Theta\}} (f_{NN}(\{\tilde{W}, \Theta\}) = E_{y, x} [l(-y F_x(\{\tilde{W}, \Theta\}))])\}$, where l $\{\displaystyle l\}$ is a loss function, $\tilde{W} = \{w^{(1)}, \dots, w^{(m)}\}$ $\{\displaystyle \{\tilde{W}\} = \{w^{(1)}, \dots, w^{(m)}\}\}$, and $\Theta = \{\theta^{(1)}, \dots, \theta^{(m)}\}$ $\{\displaystyle \Theta = \{\theta^{(1)}, \dots, \theta^{(m)}\}\}$. Consider fixed Θ $\{\displaystyle \Theta\}$ and optimizing only $\tilde{W}$ $\{\displaystyle \{\tilde{W}\}\}$, it can be shown that the critical points of $f_{NN}(\tilde{W})$ $\{\displaystyle f_{NN}(\{\tilde{W}\})\}$ of a particular hidden unit i $\{\displaystyle i\}$, $\hat{w}^{(i)}$ $\{\displaystyle \hat{w}^{(i)}\}$, all align along one line depending on incoming information into the hidden layer, such that

-- 03 --

Internal covariate shift Each layer in a neural network has inputs that follow a specific distribution, which shifts during training due to two main factors: the random starting values of the network's settings (parameter initialization) and the natural variation in the input data. This shifting pattern affecting the inputs to the network's inner layers is called internal covariate shift. While a strict definition

isn't fully agreed upon, experiments show that it involves changes in the means and variances of these inputs during training. Batch normalization was first developed to address internal covariate shift. During training, as the parameters of preceding layers adjust, the distribution of inputs to the current layer changes accordingly, such that the current layer needs to constantly readjust to new distributions. This issue is particularly severe in deep networks, because small changes in shallower hidden layers will be amplified as they propagate within the network, resulting in significant shift in deeper hidden layers. Batch normalization was proposed to reduced these unwanted shifts to speed up training and produce more reliable models. Beyond possibly tackling internal covariate shift, batch normalization offers several additional advantages. It allows the network to use a higher learning rate—a setting that controls how quickly the network learns—without causing problems like vanishing or exploding gradients, where updates become too small or too large. It also appears to have a regularizing effect, improving the network's ability to generalize to new data, reducing the need for dropout, a technique used to prevent overfitting (when a model learns the training data too well and fails on new data). Additionally, networks using batch normalization are less sensitive to the choice of starting settings or learning rates, making them more robust and adaptable.

-- 04 --

Transformation In a neural network, batch normalization is achieved through a normalization step that fixes the means and variances of each layer's inputs. Ideally, the normalization would be conducted over the entire training set, but to use this step jointly with stochastic optimization methods, it is impractical to use the global information. Thus, normalization is restrained to each mini-batch in the training process. Let us use B to denote a mini-batch of size m of the entire training set. The empirical mean and variance of B could thus be denoted as

-- 05 --

$y_i^{(k)} = \gamma^{(k)} x_i^{(k)} + \beta^{(k)}$ $\{\displaystyle y_{i}^{(k)}=\gamma^{(k)}\hat{x}_{i}^{(k)}+\beta^{(k)}\}$, where the parameters $\gamma^{(k)}$ $\{\displaystyle \gamma^{(k)}\}$ and $\beta^{(k)}$ $\{\displaystyle \beta^{(k)}\}$ are subsequently learned in the optimization process. Formally, the operation that implements batch normalization is a transform $BN_{\gamma^{(k)}, \beta^{(k)}} : x_{1...m}^{(k)} \rightarrow y_{1...m}^{(k)}$ $\{\displaystyle BN_{\gamma^{(k)}, \beta^{(k)}}:x_{1...m}^{(k)}\rightarrow y_{1...m}^{(k)}\}$ called the Batch Normalizing transform. The output of the BN transform $y^{(k)} = BN_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})$ $\{\displaystyle y^{(k)}=BN_{\gamma^{(k)}, \beta^{(k)}}(x^{(k)})\}$ is then passed to other network layers, while the normalized output $\hat{x}_i^{(k)}$ $\{\displaystyle \hat{x}_{i}^{(k)}\}$ remains internal to

the current layer.

-- 06 --

Decoupling Another possible reason for the success of batch normalization is that it decouples the length and direction of the weight vectors and thus facilitates better training. By interpreting batch norm as a reparametrization of weight space, it can be shown that the length and the direction of the weights are separated and can thus be trained separately. For a particular neural network unit with input x and weight vector w , denote its output as $f(w) = E_x [\phi(x^T w)]$, where ϕ is the activation function, and denote $S = E[xx^T]$. Assume that $E[x] = 0$, and that the spectrum of the matrix S is bounded as $0 < \mu = \lambda_{\min}(S)$, $L = \lambda_{\max}(S) < \infty$, such that S is symmetric positive definite. Adding batch normalization to this unit thus results in